BitDecoding: Unlocking Tensor Cores for Long-Context LLMs with Low-Bit KV Cache

Dayou Du[1†], Shijie Cao[2*], Jianyi Cheng[1], Luo Mai[1], Ting Cao[3], Mao Yang[2]
[1]University of Edinburgh [2]Microsoft Research
[3]Institute for AI Industry Research (AIR), Tsinghua University
{dayou.du, jianyi.cheng, luo.mai}@ed.ac.uk, {shijiecao, maoyang}@microsoft.com, tingcao@mail.tsinghua.edu.cn

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408481

***Abstract*—The rise of long-context Large Language Models (LLMs) amplifies memory and bandwidth demands during autoregressive decoding, as the Key–Value (KV) cache grows with each generated token. Low-bit KV-cache quantization (e.g., 4-bit or 2-bit) can reduce memory footprint while preserving accuracy, but existing systems suffer from slow decoding due to their exclusive reliance on CUDA cores, neglecting Tensor Cores—the primary source of compute on modern GPUs.**

We present BitDecoding, a new long-context LLMs inference system with low-bit KV cache. BitDecoding enables efficient low-bit KV cache decoding by cooperatively leveraging CUDA Cores and Tensor Cores. It introduces methods for automatically inducing optimized layouts to exploit Tensor Cores, along with novel warp-level parallelization strategies for dequantization. For unified system support, BitDecoding includes a query transformation module supporting diverse attention variants, a quantization kernel to support both tensor-wise and channel-wise scaling used in various quantization algorithms with high performance, and a dequantization kernel with a software-defined pipeline to coordinate CUDA and Tensor Cores execution for mix-precision operations. In addition, architecture-specific optimizations leverage Hopper's warpgroup tensor instructions and Blackwell's native low-precision tensor formats to maximize decoding throughput on the latest GPU generations.

Evaluated on Blackwell, Hopper, Ada, and Ampere architectures, BitDecoding attains on average a 7.5× decoding speedup over FP16 FlashDecoding-v2, and further reaches up to 8.6× with native MXFP4 formats on Blackwell, while surpassing the state-of-the-art low-bit system QServe by up to 4.3×. On LLaMA-3.1-8B with a 128K context, BitDecoding reduces single-batch decoding latency by 3×, demonstrating substantial improvements for long-context generation, and is open sourced at https://github.com/OpenBitSys/BitDecoding.

I. Introduction

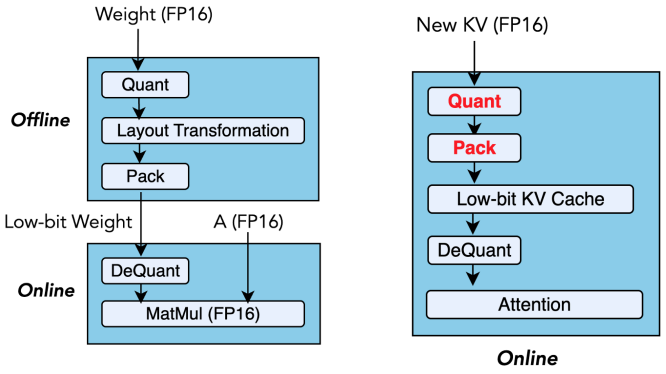
The ability of Large Language Models (LLMs) to process **long contexts** [7], [23], [30] has unlocked new capabilities, such as book summarization [4], multi-modal understanding [35], and test-time scaling [11], [22]. However, these advancements come with significant memory and computational challenges, primarily due to the large size of the Key-Value (KV) cache in long-context scenarios. During autoregressive decoding, LLMs must repeatedly access this growing cache for each generated token, which increases memory usage and slows down decoding. The problem worsens with larger batch sizes, as the KV cache scales linearly with the number of concurrent queries. For example, a 7B model requires approximately 14GB GPU memory for its parameters, but with a 32K context length and a batch size of 8, the KV cache alone consumes 128GB GPU memory [12], creating a significant memory bottleneck.

To address this growing bottleneck, **KV cache quantization** has emerged as a promising solution. By reducing the bit-width of the KV cache, quantization lowers memory overhead and improves overall efficiency. Recent quantization algorithms have shown that low-bit KV cache can retain high accuracy. QServe [16] demonstrates 4-bit KV cache improves throughput on models like LLaMA-3 and Qwen-1.5 while maintaining strong accuracy, even together with 4-bit weight and 8-bit activation. Further research [13], [18], [27] shows that 2-bit KV cache can achieve near fp16 accuracy. Kivi [18], for instance, incurs only a 0.6% accuracy drop on LongBench [3] with a 2-bit KV cache on LLaMA-2-7B-Chat. Recent studies [29], [36] explore 1-bit quantization for KV cache, maintaining acceptable accuracy under specific conditions. These results confirm that KV cache quantization strikes an effective balance between efficiency and accuracy, making it viable for long-context LLM deployment.

Despite the memory savings, current system support for low-bit KV cache struggles to deliver the expected speedup. Previous implementations [16], [18], [37] remain preliminary and case-specific, with significant room for further systematic optimization. A major bottleneck lies in the overhead introduced by quantization and dequantization. Although the KV cache is low-bit, the query (Q) values and attention scores remain in high precision. This results in mixed-precision matrix multiplications (mpGEMM), which existing hardware does not natively support, requiring dequantization before multiplication. Previous mpGEMM kernels like Ladder [33] and Marlin [9] are designed for low-bit weights but cannot be directly applied to low-bit KV caches. This is because weights are *static and stored offline*, while KV caches are *dynamic and generated online*. In autoregressive decoding, each newly generated token requires quantization, packing, and dequantization of the low-bit KV cache, introducing significant overhead and complexity in GPU kernel design, as illustrated in Fig. 1.

To address this, our insight is to leverage Tensor Cores for intensive matrix multiplications while efficiently utilizing

[†]Work partially done during an internship at Microsoft Research.
[*]Corresponding author.



(a) Low-bit weight (b) Low-bit KV cache

Fig. 1: Comparison of mixed-precision matrix multiplication for low-bit weight and low-bit KV cache. (a) Quantized weights can be preprocessed offline. (b) KV cache requires online quantization and packing for each newly generated token.

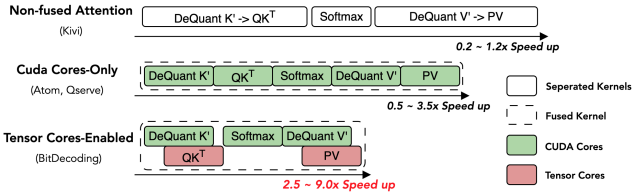


Fig. 2: Comparison of different low-bit KV cache systems against half-precision FlashAttention. Each system follows the attention formulation $\mathrm{Out} = \mathrm{softmax}(Q\,\mathcal{D}(K'^{\top}))\,\mathcal{D}(V')$, where K' and V' are low-bit quantized Key and Value tensors, and $\mathcal{D}(\cdot)$ denotes the dequantization function.

CUDA cores for KV cache dequantization. Previous work either implemented with separated kernels or fused attention operations relied solely on CUDA cores, leaving Tensor Cores underutilized, as shown in Fig. 2. Our approach is based on three key observations: First, modern language models employ Grouped-Query Attention (GQA) and Multi-Query Attention (MQA), which share a group of keys across multiple queries, enabling Tensor Cores to accelerate dot products in the self-attention mechanism. Second, leveraging Tensor Cores can alleviate computational pressure on CUDA cores, enabling more efficient execution of low-bit operations. Finally, newer GPU architectures provide distinct mechanisms: Hopper's support for asynchronous execution and warp specialization allows low-bit operations to overlap with computation [19], while Blackwell's native support for low-precision formats (e.g., MXFP4) reduces these overheads by minimizing the need for on-the-fly data conversion.

Efficiently leveraging Tensor Cores for decoding with low-bit KV caches poses significant challenges. First, Tensor Cores require dequantized low-bit data to be aligned with high-precision formats, which is difficult in autoregressive decoding as the KV cache grows dynamically and must conform to Tensor Cores-specific layouts. Without optimized layouts, Tensor Cores may exhibit poor utilization or even produce incorrect results. Second, the high cost of dequantization can stall Tensor Cores execution, reducing GPU occupancy due to mismatched workloads between CUDA cores and Tensor Cores. Third, supporting low-bit KV caches across diverse attention mechanisms and quantization algorithms—with varying tensor-wise and channel-wise scaling—demands a general yet highly optimized implementation. Without careful design, either CUDA cores or Tensor Cores become performance bottlenecks during long-context generation.

To address the above challenges, we have designed and implemented **BitDecoding**, a high-performance long-context LLMs inference system with low-bit KV cache. The design of BitDecoding delivers several contributions essential for exploiting Tensor Cores, including: (i) inducing low-bit optimized layouts based on hardware instructions, (ii) aligning warps with residual buffer to saturate Tensor Cores, (iii) remapping layouts for faster dequantization, and (iv) coordinating kernels for quantization and dequantization. In addition, we contribute new strategies for parallelizing GPU warps to mitigate low-bit operations overhead, including (i) efficient warp parallelism layout, and (ii) enhancing attention algorithms for fast warp synchronization leveraging the GPU memory hierarchy.

We further contribute implementation techniques in BitDecoding for LLMs inference, including: (i) a query transformation approach that enables efficient execution of diverse attention variants, allowing BitDecoding to be easily adopted in existing LLMs; (ii) a high-performance quantization kernel that supports both channel-wise and tensor-wise scaling, ensuring generality across quantization algorithms; and (iii) a dequantization kernel with a software-defined pipeline that coordinates CUDA and Tensor Cores for GEMM and dequantization, while overlapping data movement, including extra low-bit metadata; furthermore, BitDecoding incorporates architecture-specific optimizations that unlock Hopper's warp-group tensor operations and Blackwell's native low-precision tensor formats to maximize decoding performance on the latest GPU generations.

BitDecoding is evaluated at both the kernel and end-to-end levels across Blackwell, Hopper, Ada, and Ampere GPU architectures. At the kernel level, it outperforms FP16 FlashDecoding-v2 by up to 8.6× on Blackwell (e.g., RTX 5090, using native MXFP4 format support), 8.0× on Hopper, 7.5× on Ada, and 4.8× on Ampere, while surpassing QServe by up to 4.3×. At the end-to-end model level, BitDecoding reduces single-batch decoding latency by 3× on LLaMA-3.1-8B with a 128K sequence length and achieves over 4× higher serving throughput than QServe.

II. Background and Motivation

LLMs inference and low-bit KV cache. LLMs inference comprises two stages: (i) *Prefill*, which processes the prompt and computes Key (K) and Value (V) tensors for caching; and (ii) *Decode*, which updates the KV cache token-by-token

for autoregressive generation. For a model with n layers, h_{kv} KV heads, and hidden size d, the KV cache requires $2 \cdot 16 \cdot n \cdot h_{kv} \cdot d \cdot b \cdot l$ bits (assuming FP16), where b is the batch size and l is the sequence length. Because this requirement grows linearly with both b and l, the KV cache often dominates memory usage, especially for long-context and large-batch workloads. In batched inference, each sequence has an independent past context, so there is little batch-level parallelism or reuse when loading cached Keys and Values; *consequently, KV-cache access is typically bound by memory bandwidth*. These constraints have spurred extensive research and industrial efforts on lower-bit KV caches [12], [18], [36] to reduce memory footprint and improve throughput while preserving accuracy close to non-quantized baselines.

Tensor Cores and CUDA cores on modern GPUs. When optimizing LLM inference and low-bit KV caches on GPUs, it is crucial to exploit both Tensor Cores and CUDA cores. Tensor Cores deliver the majority of compute FLOPS in modern GPUs but are specialized for matrix operations (e.g., GEMM), whereas CUDA cores provide more flexible vector, scalar, and control-flow capabilities at substantially lower peak FLOPS. For example, on the A100, Tensor Cores deliver up to 312 TFLOPS in FP16/BF16—far exceeding the 19.5 TFLOPS FP32 offered by CUDA cores.

This performance gap has widened significantly in recent generations. The Hopper architecture introduces Warpgroup Matrix Multiply-Accumulate (WGMMA) instructions and warp-specialized pipelines to maximize asynchronous execution efficiency. The Blackwell architecture further exacerbates this disparity by supporting native micro-scaling formats (e.g., MXFP4, NVFP4), delivering up to 20 PFLOPS.

For fast LLM inference, substantial effort has gone into optimizing attention variants to exploit Tensor Cores. SOTA LLMs [10], [17], [34] increasingly adopt MQA [26] and GQA [1], which reduce memory bandwidth by reusing KV heads across multiple queries. This reuse increases arithmetic intensity and improves compute efficiency [28], aligning well with the high-throughput, matrix-centric design of Tensor Cores. Consequently, leveraging Tensor Cores is becoming essential for efficient inference in long-context and grouped-attention LLMs.

Limitations of existing low-bit KV cache systems. To support low-bit KV caches for long-context LLM inference, a number of systems have been proposed [16], [18], [37]. However, they often leave GPUs underutilized, leading to sub-optimal performance. We summarize the key reasons below.

- *Attention with separated low-bit KV-cache kernels:* The most straightforward approach, exemplified by Kivi [18], decomposes mixed-precision attention into multiple standalone kernels and embeds them in a non-fused attention implementation. This design is highly flexible and readily supports many attention variants [1], [26]. Yet the isolated launches repeatedly load and store intermediate data, inflate global-memory traffic, and break on-chip data reuse. The result is high launch overhead, increased memory bandwidth pressure, and lower effective throughput.
- *Fused attention with low-bit KV-cache kernels on CUDA cores solely:* Given the generality of CUDA cores for mixed-precision operations, a natural extension of FlashAttention-style fusion [6] is a CUDA-cores–only implementation of low-bit KV caches. While this outperforms non-fused designs, it still underutilizes Tensor Cores. In these systems, both dequantization and matrix operations (GEMV/GEMM) are executed on CUDA cores via fused multiply–add (FMA) instructions. Under mixed precision, CUDA cores must handle expensive dequantization (e.g., int4/8 $\rightarrow$ FP16/BF16), scaling, and element-wise ops—tasks that are memory-bound and consume instruction slots, register bandwidth, and L1/L2 capacity. This reduces occupancy and limits tile sizes, leaving fewer resources for the compute-heavy matrix multiplications. Consequently, running both dequantization and matmul on CUDA cores introduces significant overhead, especially for attention variants with higher arithmetic intensity.

III. Proposed Solutions and Challenges

A. Solution: Cooperative use of Tensor Cores & CUDA Cores

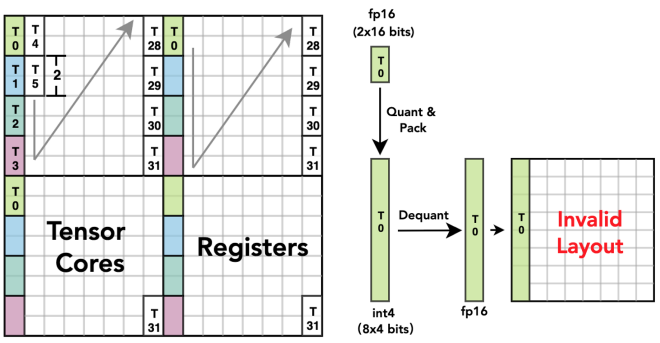
In this paper, we want to explore a solution that can achieve a *cooperative* use of Tensor Cores and CUDA cores to support low-bit KV caches during long-context LLMs inference. Our design introduces new designs and implementations that (i) construct and schedule matrix multiplications on Tensor Cores, and (ii) execute non-matrix-multiplication operations—quantization, packing and dequantization—efficiently on CUDA cores. To make this cooperation effective, we balance workloads across the Tensor Cores and CUDA cores and carefully orchestrate data movement so that dequantization feeds Tensor-Core GEMM without stalls, memory traffic is minimized, and end-to-end decoding throughput is maximized.

To ensure broad adoption, we aim to realize this cooperative design as a system that (i) supports low-bit KV caches across multiple attention variants (including MHA, MQA, and GQA), and (ii) spans multiple GPU generations. The former requires a clean interface that integrates with existing attention implementations; the latter requires designs that are easy to adapt, enabling rapid targeting of different GPU backends while sustaining high decoding throughput.

We expect significant benefits from this proposed solution. For example, by enabling low-bit decoding that builds on FlashAttention-3 (FA-3) [25], we can leverage SM90-specific features—such as warp-specialized pipelines—that yield up to $6\times$ speedups over prior implementations, avoiding the 35% throughput penalty associated with legacy SM80 instructions. Furthermore, this design anticipates the architectural capabilities of Blackwell, where native support for low-precision formats will drive even more substantial throughput improvements.

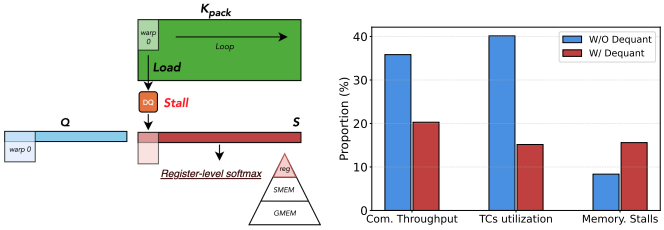
B. Open challenges

Although promising, the *cooperative* use of Tensor Cores and CUDA cores for low-bit KV caches is particularly chal-



(a) FP16 Fragment layout (b) Int4 Fragment layout

Fig. 3: (a) `mma.m16n8k16` fragment layout for matrix B. Each thread (T_i) is assigned a specific set of values based on the instruction-defined interleaved mapping. (b) For INT4, quantization packs values contiguously per thread. After dequantization, the layout misaligns with the expected interleaved pattern.



(a) Original Warp Design (b) Micro-level Analysis

Fig. 4: (a) A single warp along N for register-level operations will experience stalls due to dequantization (DQ) (b) Micro-level comparision with and without dequantization.

lenging to implement for several reasons:

Challenge 1: Tensor Cores often suffer from low-bit layout mismatches. Aligning low-bit data layouts with Tensor Cores requirements is difficult, especially in autoregressive generation where KV caches expand dynamically.

At runtime, after quantization and packing, the low-bit KV cache must dequantize into a half-precision layout that matches what Tensor Cores expect. This matching is challenging for three reasons.

First, fragment layouts vary across instructions and GPU generations. After using the optimized data-movement instruction `ldmatrix`, the fragment residing in registers enforces a strict value-to-thread mapping. Fig. 3a illustrates the registers read by each thread (T) for `mma.m16n8k16` with repeat tiling along the N dimension. However, this mapping differs from other Tensor Core instructions (e.g., `mma.m16n8k8`) and from Hopper's `wgmma` family (e.g., `wgmma.m64n64k16`).

Second, low-precision bitwidths exacerbate alignment issues. Although Tensor Cores instructions require specific compute types, their rigid, interleaved register layout makes lower-precision data hard to match directly. Without a layout transform, the low-bit register layout becomes an **invalid layout** for MMA execution due to misalignment with the interleaved access patterns. As shown in Fig. 3b, two FP16 values originally computed by Thread 0 (T0) may be quantized and packed as eight consecutive low-bit values in the KV cache; after unpacking and dequantization, they no longer align with the expected Tensor Core register layout, yielding incorrect values. Even with native low-precision formats in Blackwell, hardware support remains limited, especially for the KV cache, which still depends on continuous quantization and packing; software must therefore carefully handle low-precision values and micro-scaling factors [20].

Finally, dequantization can bottleneck execution: naive low-bit→FP16 casts are slow [14] and require a **friendly layout** to run efficiently. Prior work such as Ladder [33] and Marlin [9] mitigates mismatch for static weights by inserting separate layout-transformation kernels, but this adds substantial overhead and is unsuitable for dynamic decoding. Experimental details are given in Table II.

Challenge 2: Frequent stalls limit Tensor Cores utilization. We observe that empirically tuned warp layouts and partitioning in high-performance attention kernels often inadvertently degrade low-bit KV-cache performance.

Under FlashAttention's original warp partitioning, the additional dequantization (DQ) can substantially reduce throughput and Tensor Core utilization. As shown in Fig. 4a, FlashAttention assigns a single warp along the N dimension to perform register-level softmax and the matrix multiplication PV, with P stored in registers aligned to the Tensor Core layout. When DQ is inserted before the matmul, this strategy becomes inefficient: small warp tiles of K or V must traverse N sequentially, so DQ frequently stalls the warp. Nsight Compute profiling [21] in Fig. 4b confirms that the added DQ overhead increases memory-access stalls and depresses compute throughput and Tensor Cores utilization, consistent with prior observations [8].

Furthermore, native low-precision formats introduce their own overhead despite eliminating dequantization. Specifically, to utilize low-precision Tensor Cores for the second matrix multiplication (PV), the probability matrix P must be dynamically re-quantized after the softmax operation: $P_{f16} = \mathrm{softmax}(Q_{f4}K_{f4}^T), \quad O_{f16} = \mathbf{Quant}(P_{f16})V_{f4}$. This on-the-fly quantization creates a new computational bottleneck that can similarly stall Tensor Cores execution.

Challenge 3: Lack of generalizable system optimizations for different low-bit KV-cache methods. Popular KV-cache quantization methods use diverse scaling granularities for the Key tensor—tensor-wise [12], [37] and channel-wise [13], [18]—which complicates building a unified system that supports them all. Online quantization and packing require reductions and element-wise transforms, adding nontrivial runtime overhead. Moreover, auxiliary metadata (scale and zero-point) increases memory traffic and, without careful scheduling, disrupts the load–compute pipeline. Prior mixed-precision kernel optimizations [9], [33] target static weight quantization and

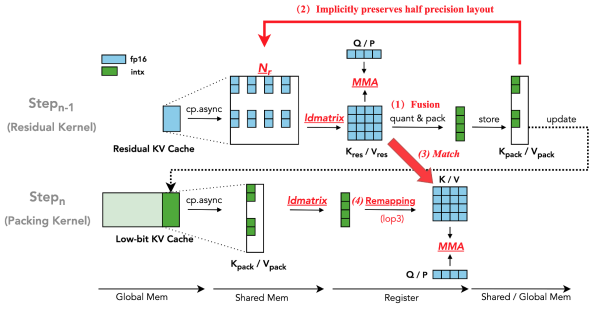


Fig. 5: Overview of methods for optimizing low-bit layout on Tensor Cores. (1) Fused computation and quantization within Tensor Cores fragments. (2) The low-bit packing data preserves FP16 values. (3) Low-bit Layout matches with the dequantized half-precision layout. (4) Layout remapping for faster dequantization.

do not generalize to the dynamic, step-by-step nature of KV caches. To date, generalizable system-level optimization techniques for high-performance, low-bit KV-cache quantization are lacking.

IV. BitDecoding Design

In this section, we present the design of BitDecoding system which realizes the cooperative use of Tensor Cores and CUDA cores in supporting low-bit KV cache. The design primarily contains (i) new methods and principles for optimizing the low-bit layout in using Tensor Cores, and (ii) new strategies for parallelizing and coordinating GPU warps that can minimize the stalls due to dequantization.

A. Methods for optimizing low-bit layout on Tensor Cores

The first challenge our design aims to address is to ensure BitDecoding can automatically generate an optimized layout that can fully utilize Tensor Cores across different GPU generations and different configurations of the low-bit KV caches. For this, we have designed the following principles and methods:

(1) Inducing low-bit optimized layout with hardware instructions. Our design is motivated by a novel insight: the thread-to-register mapping of `ldmatrix` loads data in Tensor Core's interleaved fragment layout. As shown in Fig. 5-(2), if each thread then quantizes and packs locally, the resulting low-bit packing *implicitly preserves* the half-precision (FP16) interleaved layout. On unpacking and dequantization, values already match Tensor Core registers—no global reshape is required. Thus, rather than relying on heavyweight global transforms via manual implementations [9] or iterative search [33] as in prior methods, we use hardware instructions to automatically induce a valid low-bit packing layout while computing. This yields zero-overhead remapping that is efficient, compatible with Tensor Cores execution, and avoids extra data movement.

Building on this insight, we design a dedicated GPU *Residual Kernel* that fuses computation, quantization, and packing for newly generated FP16 KV tensors. Using `ldmatrix`, we load the high-precision KV tensor into registers structured for Tensor Cores, perform the matrix operation (e.g., $QK^\top$ or PV), and then have each thread quantize and pack its portion in registers (see Fig. 5-(1)). The result is interleaved, layout-compatible low-bit data written directly to global memory, updating the low-bit KV cache.

To consume this cache, we introduce a *Packing Kernel* that fuses dequantization with computation. To guarantee correct register layout during unpacking, it mirrors the Residual Kernel's instruction configuration which (i) uses the same `ldmatrix` variant and (ii) follows the same `mma` variant and warp-tiling configuration. Consequently, when the Packing Kernel loads packed low-bit data via `ldmatrix`, the unpacked values are inherently aligned with Tensor Core registers and can participate in matrix multiplication immediately, without explicit layout correction.

(2) Aligning warps with residual KV cache to saturate Tensor Cores. Tensor Cores execute warp-tiled matrix operations, which require input tiles to be fully populated to achieve optimal throughput. Based on this, *our insight* is that by allocating a residual buffer with size matching the tiling capacity of Tensor Cores, we ensure that low-bit data aligns with the compute granularity of the hardware to fully utilize the computing ability of the computing unit.

To implement this idea, we introduce a half-precision residual KV cache with a residual block size N_r. Let $X \in \mathbb{R}^{L\times d}$ denote the entire KV cache. We partition X into:

$$X = X_{\text{pack}} \cup X_{\text{res}}, \quad \text{where} \quad \begin{cases} X_{\text{pack}} = X[:L - N_r] \\ X_{\text{res}} = X[L - N_r :] \end{cases}$$

We define β as the bit-width for low-bit quantization (e.g., $\beta = 4$ or 2), and ω as the word size used for packed storage (e.g., $\omega = 16$ for INT16). The corresponding *packing ratio* is given by $R = \omega/\beta$. Let W_n denote the number of warps along the N dimension, and P_n the number of elements each warp tile processes (e.g., $P_n = 8$ under `mma.m16n8k16`). To ensure each Tensor Cores fragment is fully populated for each warp, the residual block size is computed as:

$$N_r = P_n \times W_n \times R \tag{1}$$

This guarantees that low-bit KV cache fragments align precisely with the warp-level tiling of Tensor Core operations, enabling dense, layout-compatible packing and maximizing compute unit occupancy.

(3) Re-mapping layout for faster dequantization. Though compatible with Tensor Cores layout, the layout is inefficient to dequantization due to directly casting low-bit values to FP16 using `static_cast` introduces significant overhead.

To mitigate this inefficiency, we further design a faster dequantization mapping approach based on low-level bitwise operations and instructions inspired by [14]. After loading packed data into registers using `ldmatrix`, we cast them to INT32 before mapping them to the interleaved Tensor Core layout following the 75316420 pattern. This layout enables

efficient conversion of INT4/INT2 data to FP16 using the `lop3` instruction for bitwise manipulation while aligning with the Tensor Core computation pattern.

(4) Coordinating Residual and Packing Kernels with Configuration Setup. This design is executed by coordinating the Residual and Packing kernels under a unified instruction configuration. First, the hardware instruction configuration—including `ldmatrix` and `mma` variants—can be determined based on GPU architectures. With this configuration, the residual block size N_r is computed based on the bit-width of the low-bit KV cache. As shown in Fig. 5, the Residual kernel loads high-precision KV entries into registers via `ldmatrix`, performs computation using Tensor Cores, and then fuses quantization and packing before storing the results into the low-bit KV cache. The Packing kernel, using the same instruction configuration, loads the packed data into registers, performs efficient dequantization, and proceeds with Tensor Core computation.

B. Strategies for parallelizing warps

The second challenge is ensuring BitDecoding avoids the pitfalls of existing warp-parallelization strategies for mixed-precision attention, which suffer from low hardware utilization due to frequent warp stalls. Our key insight is that low-bit data moves at much higher bandwidth than full precision, shifting the bottleneck from memory to compute. We therefore design a warp layout that exploits the GPU memory hierarchy to parallelize low-precision operations efficiently, minimizing data movement and substantially improving Tensor Cores utilization (Table III demonstrates minimal overhead).

(1) Enhancing warps parallelism for low-precision operations. We introduce a novel warps layout to enable parallel operations of multiple packed data chunks. Using dequantization as an example, we modify the warp partitioning strategy to better exploit parallelism. As illustrated in Fig. 6, instead of the original strategy that allocates multiple warps along the M dimension, we constrain the allocation to $W_m = 1$—leveraging the fact that the decoding query length is typically small (< 16)—and reallocate resources to increase the number of warps along the N dimension (W_n).

By increasing W_n, dequantization stalls can be effectively mitigated by the Streaming Multiprocessor (SM) warp scheduler [24], as multiple warps concurrently execute dequantization on packed data before proceeding to Tensor Cores-based matrix multiplication.

Similarly, this parallelism strategy alleviates the stalls introduced by on-the-fly quantization in native low-precision attention, ensuring that neither quantization nor dequantization becomes a serialization bottleneck.

(2) Leveraging memory hierarchy for warps synchronization. However, with results now distributed across different registers and warps, the original register-level softmax becomes infeasible. Moreover, *a key challenge emerges* due to the incompatibility between the new warp layout and the expected format for MMA operations on PV.

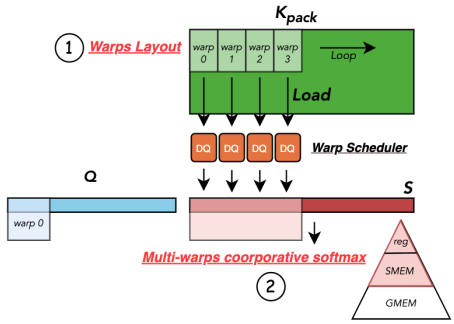


Fig. 6: Enhancing parallism for efficient Tensor Cores utilization with (1) new warp layout design reduces dequantization stalls and (2) cooperative softmax leverages data movement between GPU register and shared memory for cross-warp reduction with minimal overhead.

To address this, we leverage a multi-level memory hierarchy—spanning registers and shared memory—to enable cross-warp reduction and synchronization for the softmax computation. As illustrated in Algorithm 1, we extend existing high-performance attention algorithms, such as FlashAttention, by introducing two additional shared memory buffers: $sTMP \in \mathbb{R}^{W_n}$ and $sAcc \in \mathbb{R}^{T_m \times T_n}$. The buffer $sTMP$ facilitates cross-warp reduction for computing the row-wise maximum during softmax. This is achieved by first performing intra-warp reduction within registers, followed by inter-warp reduction via shared memory. The buffer $sAcc$ temporarily stores the attention scores P computed in Tensor Core registers and later reloads them via `ldmatrix`, ensuring proper alignment for subsequent Tensor Core `mma` operations.

Since W_n is typically small, we reuse the shared memory pointer of $sTMP$ for $sAcc$ to minimize memory overhead. Moreover, on Hopper Tensor Cores, WGMMA supports direct shared memory access, eliminating the need for explicit data movement from shared memory to registers.

Algorithm 1 Multi-warps Cooperative Softmax

Require: $sTMP \in \mathbb{R}^{W_n}$ and $sAcc \in \mathbb{R}^{T_m \times T_n}$ in SMEM.
Require: Load $Q_i \in \mathbb{R}^{T_m \times d}$ and $K_i, V_i \in \mathbb{R}^{T_n \times d}$ to REG.
1: $S_i = Q_i K_j^T$ where $S_i \in \mathbb{R}^{T_m \times T_n}$.
2: $m_i^{new} = \max(m_i, \text{rowmax}(S_i, sTMP))$.
3: $P_i = \exp(S_i - m_i^{new})$ where $P_i \in \mathbb{R}^{T_m \times T_n}$.
4: $sAcc = \text{tiled\_copy\_r2s}(P_i)$.
5: $P_i' = \text{tiled\_copy\_s2r}(sAcc)$
6: $O_i^{new} = P_i' V_j + \text{diag}(e^{m_i - m_i^{new}}) O_i$.

V. System Implementation

In this section, we describe how we implement BitDecoding, as illustrated in Fig. 7. Our implementation consists of three major components: (i) a *query transformation* component that supports diverse attention variants in LLMs; (ii) a *Residual Kernel* that performs low-cost quantization and packing while remaining general to both tensor-wise and channel-wise scaling across quantization algorithms; and (iii) a *Packing Kernel*

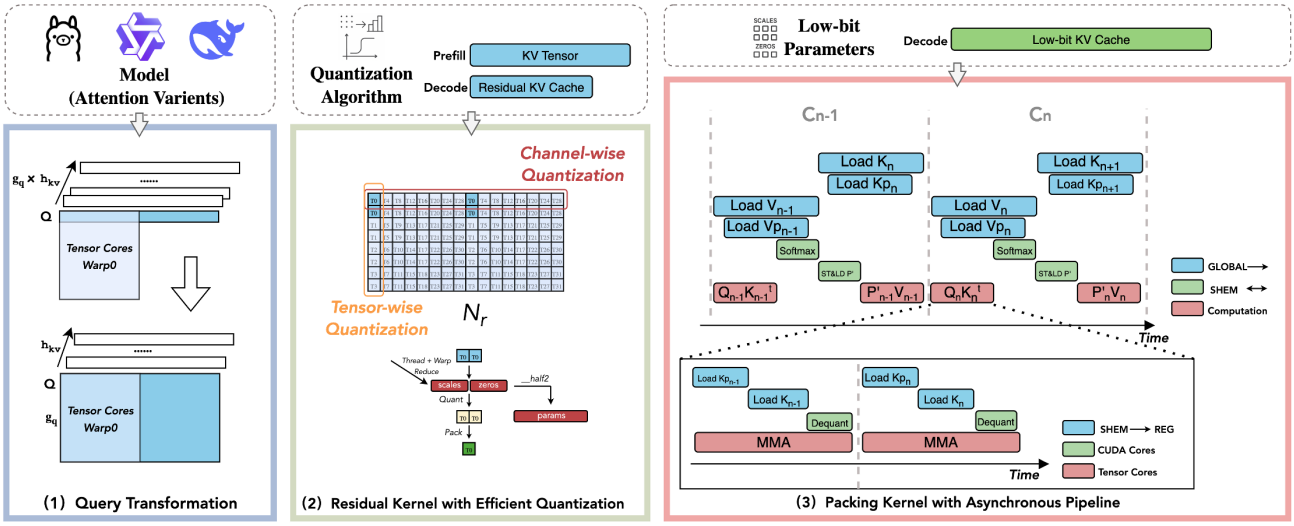


Fig. 7: System overview of BitDecoding. (1) **Query Transformation** restructures the query tensor layout to enable efficient warp-level execution for attention variants on Tensor Cores. (2) **Residual Kernel** performs quantization and packing with minimal overhead, supporting both tensor-wise and channel-wise scaling. (3) **Packing Kernel** executes dequantization and matrix multiplication using a fine-grained, asynchronous pipeline, maximizing Tensor Cores and CUDA Cores utilization with low-bit parameters.

with a fine-grained pipeline that fully utilizes both Tensor Cores and CUDA cores. Finally, we discuss architecture-specific optimizations that leverage the advanced features of the latest GPU generations (e.g., Hopper and Blackwell) to further enhance decoding throughput.

A. Query Transformation

Modern LLMs adopt diverse attention variants [10], [17], [34] with different key–value (KV) sharing patterns. BitDecoding aims to support all these variants.

For instance, in GQA and MQA, multiple query heads share a KV head, reducing the number of KV projections and memory accesses. The degree of sharing is measured by $g_q = h_q/h_{kv}$, where h_q and h_{kv} are the numbers of query and KV heads, respectively: $g_q = 1$ corresponds to MHA, $g_q > 1$ denotes GQA, and $h_{kv} = 1$ (i.e., $g_q = h_q$) characterizes MQA.

A challenge arises in decoding: since $Q\_len = 1$ (one token at a time), the query tensor has a very small batch dimension, and a naive $Q \cdot K^\top$ underfills Tensor Cores, yielding poor warp occupancy and low throughput.

To address this, we perform a *query transformation* that reorganizes the query layout to better match Tensor Core tiling. As illustrated in Fig. 7 (left), we reshape the query tensor from $[1, (g_q, h_{kv})]$ to $[g_q, h_{kv}]$, effectively forming a larger Q tile without changing the semantics of attention or its KV-sharing pattern. Grouped query heads are then processed in parallel as a larger GEMM block, fully populating Tensor Core fragments, improving warp occupancy, and increasing throughput.

B. Residual Kernel

A primary challenge in low-bit KV-cache design is supporting diverse quantization algorithms—especially differing scaling granularities (e.g., tensor-wise, channel-wise)—without sacrificing performance. Quantization involves reductions and element-wise operations to compute scale and zero-point, followed by bit-packing; during decoding these must run online, adding runtime overhead and risking misalignment with the rigid layouts expected by Tensor Cores. To address this, we design the *Residual Kernel* with two key optimizations:

(1) Partitioning KV cache based on residual block size. During prefill with context length L, we split the KV cache based on a Tensor Cores-aligned residual block size N_r (see Eq. 1). The first $N_p = L - (L \mod N_r)$ entries are quantized and packed into the low-bit KV cache using a fused quantization and packing operation. The remaining KV Tensor with size $\texttt{res\_len} = L \mod N_r$ are stored in the half-precision residual KV cache. At each decode step, the newly generated K, V tensors are appended to the residual cache and used for attention computation. This cache grows incrementally until it reaches the residual block size N_r. Once per token generation, the Residual Kernel computes attention using the half-precision residual KV cache and optionally quantizes it (when $\texttt{res\_len} = N_r$) into packed format.

With this KV cache partitioning during decoding, we can naturally perform channel-wise quantization along the seq_len and tensor-wise quantization along the hidden dimension within the residual block.

(2) Optimizing reduction with warp-level instructions. As shown in Fig. 7 (mid), once the half-precision KV data is computed, it remains in registers as Tensor Cores frag-

ments—structured in the native interleaved layout used by `mma` operations. To efficiently compute the quantization parameters (scale and zero-point), we first perform thread-level reductions to obtain local min/max statistics within each group.

These local results are then aggregated across the warp using the PTX instruction `__shfl_xor_sync`, enabling efficient warp-level reduction without shared memory. When the warp repetition factor $W_n > 1$, we introduce a small shared memory buffer to coordinate the final reduction across warps.

After computing the quantization parameters, each thread performs in-register quantization and packs the low-bit values into INT16 format. This avoids extra memory movement and keeps data in a computation-ready state. To minimize overhead, both the scale and zero-point are stored in a compact `half2` format, enabling efficient memory access and fused multiply-add during dequantization in the decode phase.

C. Packing Kernel

Another challenge is the auxiliary low-bit metadata (scale and zero-point), which increases memory traffic, while dequantization still runs on CUDA cores. Without careful scheduling, this disrupts the load–compute pipeline and prevents overlap with Tensor Core operations. We therefore design a fine-grained asynchronous pipeline: CUDA cores handle dequantization, Tensor Cores execute matrix multiplications, and both are orchestrated to overlap with memory transfers through the GPU hierarchy—enabling efficient mixed-precision computation.

(1) Optimizing asynchronous data movement. *From Global to Shared Memory*, we follow FlashAttention [6] via block-wise tiling [32] and strategic recomputation. It processes input matrices $Q \in \mathbb{R}^{T_m \times d}$, $K, V \in \mathbb{R}^{T_n \times d}$ in tiles within shared memory, using block sizes T_m and T_n. The number of key-value tiles is $C_n = \lceil L/T_n \rceil$.

To efficiently manage quantization parameters, we introduce dedicated shared memory buffers for quantization paramenter K_{pack} params (K_p) and V_{pack} param (V_p), facilitating efficient tiling for memory copy. These buffers store `scale` and `zeros` in the `half2` format, allowing them to be loaded in a single instruction.

The shape of K_p is determined by the quantization granularity setting, and the V_p follows a Tensor-wise layout:

- **Channel-wise:** $(T_n/\text{group\_size}, d)$.
- **Tensor-wise:** $(T_n, d/\text{group\_size})$.

To achieve optimal memory overlapping, all global-to-shared memory transfers are executed asynchronously using the `cp.async` intrinsic, ensuring efficient pipeline execution, as shown in Fig. 7 (right). We optimize memory transactions using instructions with different caching strategies:

- **`cp.async.cg`**: Used for Q, K_{pack}, and V_{pack}, which cache only in global memory as they are not reused within the same kernel.
- **`cp.async.ca`**: Applied to K_p and V_p, ensuring smaller byte-level alignment for fine-grained memory access.

In Hopper architecture, we follow FA3, leveraging the `tma.copy` instruction for data loading. This facilitates warp-specialized scheduling, improving data locality and reducing memory latency across multiple warps.

From Shared Memory to Register, we use the PTX instruction `ldmatrix` to efficiently load K_{pack}, V_{pack} and $sAcc$ from shared memory into registers with the Tensor Cores tiling layout. To eliminate bank conflicts, we use a sizzling scheme [5] defined as:

$$\text{col}_{id} = \text{row}_{id} \oplus \text{col}_{id} \tag{2}$$

achieve bank conflict-free access. Additionally, we restructure the shared memory layout of K_p and V_p to further reduce bank conflict and maximize throughput efficiency.

(2) Asynchronous pipeline for overlapping CUDA Cores and Tensor Cores. To fully utilize both CUDA cores and Tensor Cores, we implement a register-level, asynchronous pipeline that overlaps computation with memory operations. In this pipeline, shared-memory loads via `ldmatrix` and dequantization (`Dequant`) run concurrently with Tensor Core matrix multiplications (`mma`) under the SM warp scheduler.

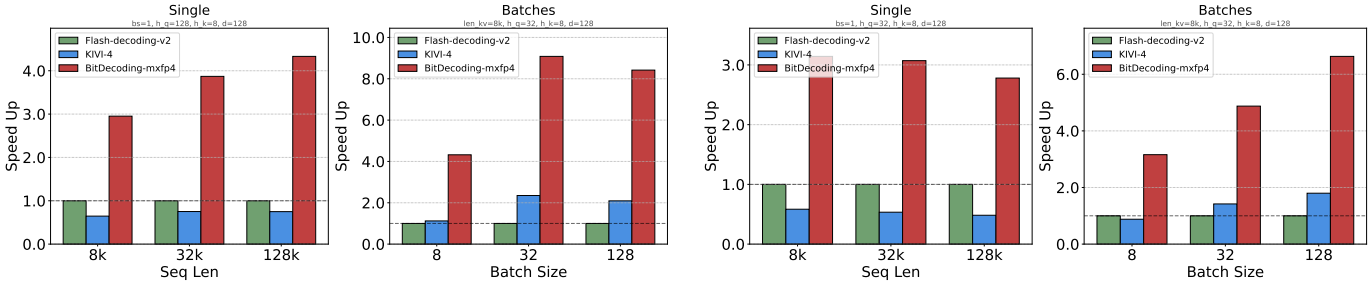
As shown in Fig. 7 (right), while the i-th slice is being processed by `mma` on Tensor Cores, the $(i+1)$-th slice is simultaneously loaded from shared memory (`ldmatrix`) and dequantized. This sustains a continuous producer–consumer flow, improving instruction throughput and maximizing utilization of both CUDA cores and Tensor Cores.

D. Latest Architectures Support

While the design presented thus far effectively targets pre-Hopper architectures (e.g., Ampere), newer generations introduce distinct hardware features that require tailored optimization strategies. Below, we detail how our approach adapts to leverage the specialized instructions and native data formats of the Hopper and Blackwell architectures.

(1) Unlocking Hopper for warpgroup acceleration capabilities via smart uses of PTX-level instructions. Hopper Tensor Cores, increasingly introduce Warpgroup Matrix Multiply-Accumulate (`wgmma`) instruction. This instruction however imposes a key constraint: in a matrix multiplication $C = AB$, only A and C can be sourced from registers, while B must reside in shared memory. This presents a challenge for low-bit quantized data, as values are typically upconverted to FP16 in registers before computation. To resolve this, we leverage Hopper's `STSM` PTX instruction to store dequantized FP16 values in shared memory efficiently, accessible for `wgmma_SS` operations. Remarkably, the asynchronous nature of WGMMA overlaps storage with computation, optimizing performance.

(2) Accelerating Blackwell with native low-precision format. The Blackwell architecture introduces native support for low-precision tensor operations, eliminating the need for explicit dequantization. Consequently, the `lop3`-based register remapping described earlier is bypassed in favor of direct execution. We target Blackwell's low-precision `mma` instructions—specifically those supporting the micro-scaling



(a) RTX 5090 (b) RTX PRO 6000

Fig. 8: Kernel performance with mxfp4 on Blackwell architectures.

formats (e.g., `mxfp4 / nvfp4`)—to execute GEMM operations directly on packed 4-bit data. While these instructions enforce rigid layout constraints for both the packed values and their block-scaling factors, the layout transformation strategy proposed in Section IV-A is designed to be layout-agnostic. It automatically aligns the packed KV data with the hardware-mandated format, ensuring seamless integration with Blackwell's native tensor pipelines.

VI. Evaluation

In this section, we comprehensively evaluate BitDecoding against state-of-the-art approaches and systems. Our evaluation highlights the following key results:

1) BitDecoding outperforms FP16 FlashDecoding-v2 by significant margins across GPU generations, achieving speedups of up to 8.6× on Blackwell (using native MXFP4), 8.0× on Hopper, and 7.5× on Ada architectures, while surpassing the state-of-the-art low-bit system QServe by up to 4.3× (Section VI-A).
2) In end-to-end long-context inference, BitDecoding reduces single-batch latency by 3x (on LLaMA-3.1-8B with 128K context) and achieves over 4x higher serving throughput than QServe, demonstrating superior scalability in GQA settings where prior CUDA Core-only methods degrade (Section VI-B).
3) BitDecoding preserves near-FP16 accuracy while deriving significant performance gains from each system component, demonstrating only a 0.2% accuracy degradation with 4-bit quantization, while our ablation study confirms that every design module contributes to the overall speedup (Section VI-C).

A. Kernels Performance Across GPU Architectures

Kernels Settings. Since different LLM serving scenarios require varying workloads and attention kernel designs, we evaluate performance under the following three representative settings:

- **Single:** A scenario where batch_size = 1, representing inference for edge users with long context.
- **Batches:** A setting with a larger batch_size, maintaining the same input length while applying simple padding.

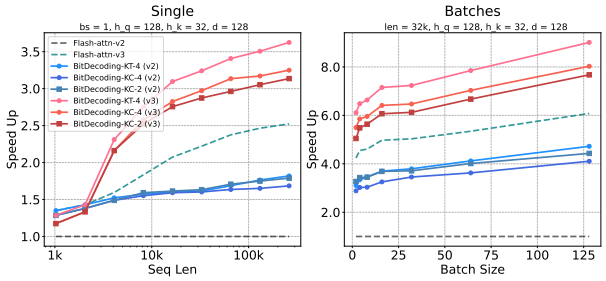


Fig. 9: Kernel performance on Hopper (H100).

- **Page:** A high-throughput scenario where a larger batch_size is managed using the page management technique [15].

Baselines. We compare BitDecoding against several representative attention kernel implementations. For FP16 KV cache, we use FlashDecoding [6], [25]—a split-partitioned variant of FlashAttention optimized for long-context decoding—as our baseline for speedup normalization. For low-bit KV cache, we evaluate Kivi [18], a non-fused kernel supporting 4-bit and 2-bit quantization; Atom [37] and QServe [16], both fused-kernel implementations with CUDA Cores-only approach and supporting 4-bit cache with page management. Notably, Atom does not support GQA.

Quantization Settings. We evaluate BitDecoding under various quantization configurations, supporting 4-bit and 2-bit Key tensors with both Channel-wise (KC) and Tensor-wise (KT) schemes.

Results on MXFP4 (RTX5090, RTX PRO 6000). The Blackwell architecture provides native support for low-precision data formats, eliminating on-the-fly dequantization overhead while delivering very high compute throughput on low-bit operations. As shown in Fig. 8a, BitDecoding achieves remarkable performance, reaching up to 8.6× speedup in batched scenarios and over 4.3× in single-batch long-context decoding (128k), significantly outpacing the non-fused attention baseline. Similarly, Fig. 8b demonstrates that the RTX PRO 6000 attains substantial gains, peaking at 6.5× speedup with large batch sizes.

Results on Advanced Tensor Cores Acceleration (H100). Newer GPU architectures often introduce advanced compute

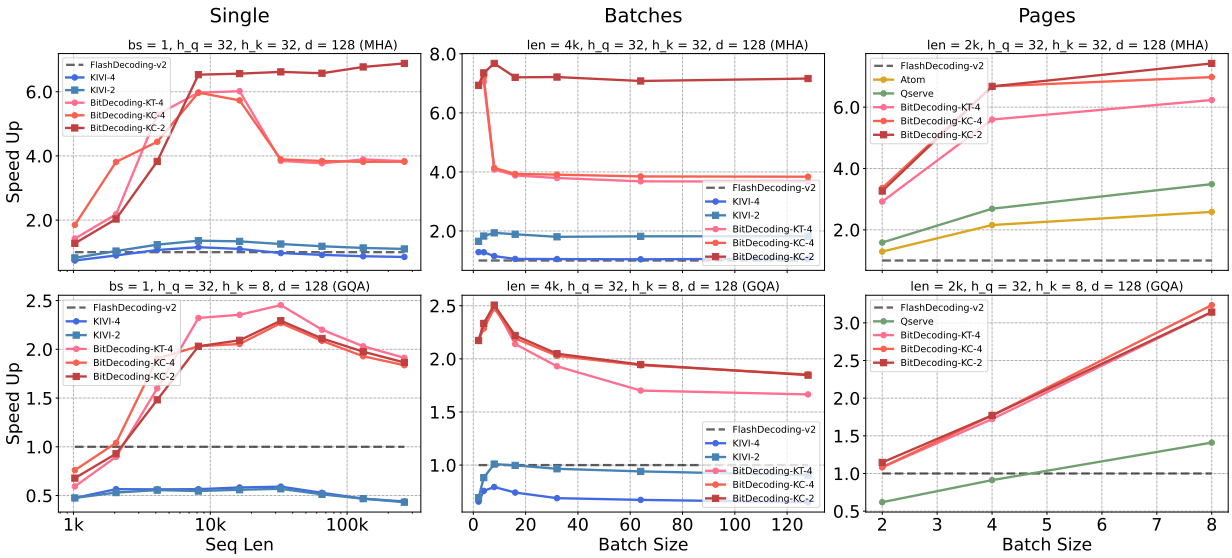


Fig. 10: Kernel performance on RTX4090.

instructions that significantly accelerate kernel execution. As illustrated in Fig. 9, FlashDecoding-v3, optimized for Hopper Tensor Cores, delivers notable performance gains over its v2 counterpart. While BitDecoding-v2 reaches up to 4.1× speedup, the v3 implementation further boosts performance to 8.0×. This is enabled by BitDecoding's use of Hopper's `wgmma` and asynchronous memory instructions, ensuring high Tensor Cores utilization even in mixed-precision settings.

Results on Bandwidth-constrained GPU (RTX 4090). Leveraging low-precision data is critical for accelerating inference on bandwidth-constrained GPUs. As shown in Fig. 10, BitDecoding achieves roughly 4× (4-bit) and over 7× (2-bit) speedups over FlashDecoding-v2 in Single and Batches settings, gains that stem directly from alleviating DRAM bottlenecks via low-bit KV caching.

BitDecoding significantly outperforms baselines across all scenarios; unlike the non-fused KIVI, which relies on separate kernels and suffers severe degradation in GQA, BitDecoding's fully fused design maintains high efficiency. In Page settings, it surpasses fused CUDA-core baselines: for MHA, BitDecoding achieves over 6× speedup compared to QServe's 3.5×. Crucially, in compute-intensive GQA, it maintains a 3× speedup while QServe drops to 1.4×, confirming that leveraging Tensor Cores provides robust acceleration where CUDA-only approaches falter.

Results on High-Bandwidth GPU (A100). On architectures with high memory bandwidth like the A100, computation pressure becomes more pronounced, as performance bottlenecks shift from memory access to compute utilization—especially when kernel designs fail to fully exploit available compute resources. As shown in Fig. 11, both KIVI and QServe suffer from poor performance—KIVI due to its non-fused kernel design, and QServe due to underutilization of Tensor Cores—even performing worse than the FP16 baseline. In contrast, BitDecoding consistently outperforms all baselines across workloads, achieving up to 3× speedup, thanks to its efficient utilization of Tensor Cores and fused execution pipeline. An interesting observation is that the performance gap between 4-bit and 2-bit variants narrows on A100, as the increased DRAM bandwidth reduces memory bottlenecks and shifts the performance balance toward compute-bound execution.

B. Performance across LLMs Inference Systems

Model settings. We evaluate on a range of LLMs, including LLaMA-2-7B, LLaMA-3.1-8B, LLaMA-3.1-70B, Qwen3-8B, and Qwen3-14B. Among them, only LLaMA-2-7B adopts MHA, while the others use GQA. All models are run on a single A100 GPU, except LLaMA-3.1-70B, which is evaluated on 8×A100 GPUs.

Quantization settings. We choose channel-wise quantization for LLMs KV cache as it brings better accuracy and aligns with the Kivi.

Compared with Non-fused Attention. As illustrated in Fig. 12, in the Single setting, BitDecoding achieves up to 3.3× speedup at a 128K context length, where KV cache loading becomes the dominant bottleneck in LLMs inference. In contrast, Kivi suffers from limited scalability and encounters out-of-memory (OOM) failures at 128K due to the lack of block-tiling kernel support. For the Batches setting, BitDecoding significantly outperforms KIVI in throughput: BitDecoding-KC-4 and KC-2 reach up to 900 and 1200 tokens/s, respectively, while KIVI-4 and KIVI-2 peak below 700 tokens/s.

Compared with CUDA Cores-only fused Attention. We compare BitDecoding with Qserve for page-setting inference, as Qserve supports both MHA and GQA attention structures. The maximum throughput is evaluated under the largest batch sizes available within GPU memory. As illustrated in Fig. 13, Qserve achieves higher throughput than FlashDecoding-v2 on LLaMA-2-7B but suffers from degraded performance on

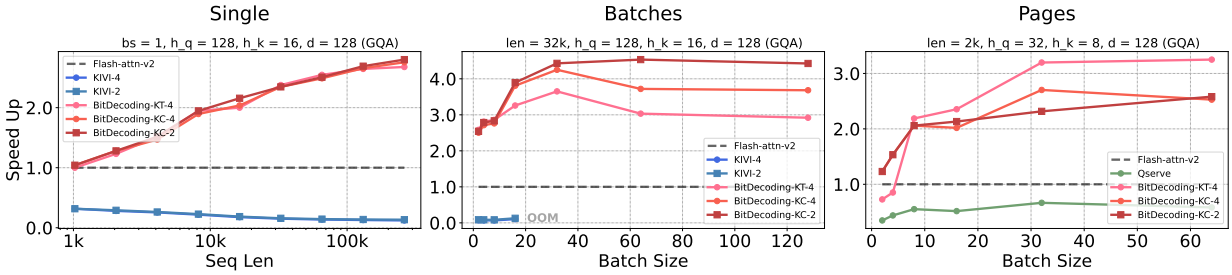


Fig. 11: Kernel performance on A100.

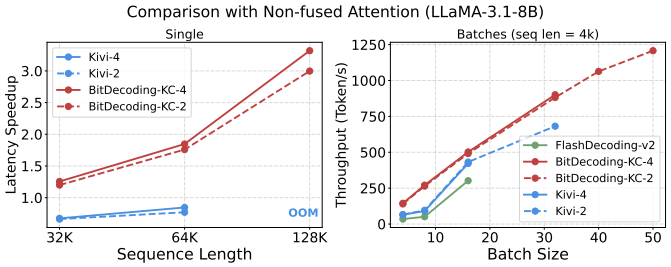


Fig. 12: Comparing Kivi with (a) end-to-end generation time and (b) decoding throughput.

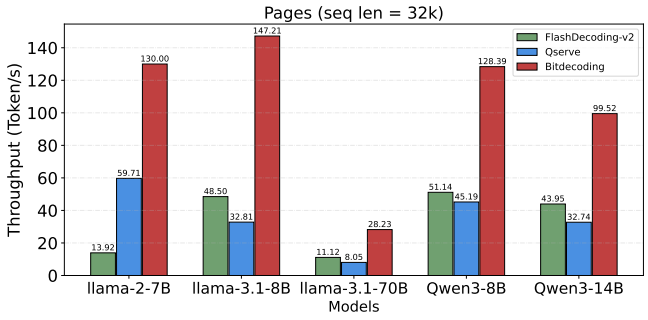


Fig. 13: Comparing Qserve with decoding throughput.

all other models due to inefficiencies in handling GQA. In contrast, BitDecoding consistently outperforms QServe across both LLaMA and Qwen architectures, under both single-GPU and multi-GPU settings, achieving more than 2× higher maximum throughput compared to QServe.

C. Accuracy, Overhead and Performance Breakdown

Accuracy analysis. As shown in Table I, we evaluate throughput and accuracy across different bit widths. The 2-bit quantization reduces memory consumption significantly, enabling larger batch sizes and achieving a $4.25\times$ higher throughput compared to FP16. Meanwhile, the 4-bit quantization achieves a $2.98\times$ speedup while maintaining near full-precision accuracy with only a minimal 0.2% degradation. These results highlight the trade-off, with 4-bit quantization offering balance and 2-bit maximizing throughput at a slight accuracy cost.

Half-precision Residual Kernel Overhead. Half-precision residual KV Cache would introduce quite a small portion

TABLE I: Efficiency and accuracy tradeoff with low-bit KV cache. We use Llama-3.1-8B-Instruct with $seq\_len = 32K$, and evaluate average accuracy on longbench [3].

KV Cache	Throughput	Longbench Acc
FP16	49.25	48.25
INT4	147.21 (+2.98x)	48.16 (-0.2%)
INT2	209.48 (+4.25x)	47.38 (-2.7%)

TABLE II: Latency (ms) comparison of quantization and packing during inference.

Inference Phase	Marlin	Ladder	BitDecoding
Prefill	58.02	4.79	0.0599
Decode	0.41	0.65	0.008

TABLE III: Impact of cooperative softmax and warps on performance and validity.

W_n	Coop. Soft	Latency (ms)	TCs Utilization (%)	Valid
1	×	3.746	10.91	✓
4	×	0.610	19.71	×
4	✓	0.613	19.66	✓

memory overhead as $seq\_len >> N_r$, while seq_len would be more than 32K and N_r is always less than 256. The half-precision residual KV cache introduces only a slight runtime overhead due to an extra kernel launch, as shown in Fig. 14. Moreover, this overhead becomes increasingly negligible as the sequence length grows, since the residual portion constitutes a smaller fraction of the total KV cache.

Quantization and Packing Overhead. We evaluate the latency of quantization and packing under a sequence length of $seq\_len = 128K$, comparing BitDecoding with Marlin [9] and Ladder [33]. As shown in Table II, the pre-transformation and packing step in previous mixed-precision computing methods introduce significant overhead, which cannot be ignored. Our kernel incurs minimal overhead after the Prefill phase, primarily due to kernel launch overhead. Moreover, during decoding, we achieves nearly negligible overhead, as it is fully fused into kernel computation.

Dequantization Overhead. Fig. 15a illustrates the high computational overhead of dequantization in Atom and QServe, consuming nearly half the kernel execution time. In

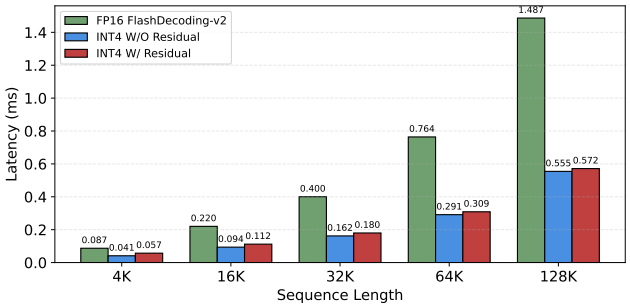
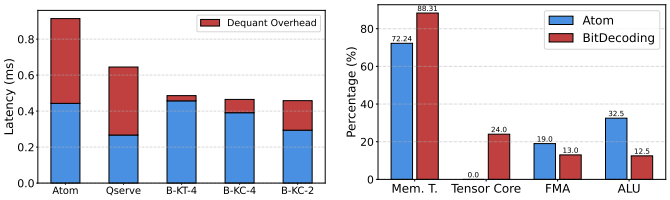


Fig. 14: Runtime overhead of the residual KV cache.



(a) Dequantization Overhead (b) Micro Analysis

Fig. 15: Dequantization overhead analysis.

contrast, BitDecoding significantly reduces this overhead to less than 15% (4-bit) and 35% (2-bit), thanks to better Tensor Cores overlap.

A further microbenchmark comparing Atom and BitDecoding (Fig. 15b) reveals BitDecoding's superior memory throughput from effective Tensor Core usage. Conversely, Atom relies heavily on CUDA cores, increasing pressure on FMA and ALU operations.

Multi-warps Cooperative Softmax Overhead. Table III shows that increasing W_n improves Tensor Cores utilization and reduces latency, but breaks correctness without cooperative softmax. Enabling cooperative softmax restores correctness with only 0.5% overhead. Although it introduces shared memory access, the overhead is minimal since low-bit data reduces memory bandwidth pressure and shifts the kernel from memory-bound to compute-bound.

BreakDown Analysis. To further analyze the performance gains of BitDecoding, we decompose our optimizations in Fig. 16. Following [2], we use a continuous-packing baseline that quantizes and packs the KV cache at every generation step, which introduces substantial overhead and requires manual effort to maintain valid layouts. In contrast, our layout design automatically induces Tensor Core–compatible layouts for arbitrary low-bit formats, fully unlocking the compute potential of Tensor Cores. On top of this, the warp-parallelism strategy contributes significant additional speedups, while the pipeline optimizations further enhance end-to-end performance.

VII. Related Works

a) KV Cache Quantization Algorithms: KV cache quantization reduces memory usage in LLMs with long contexts

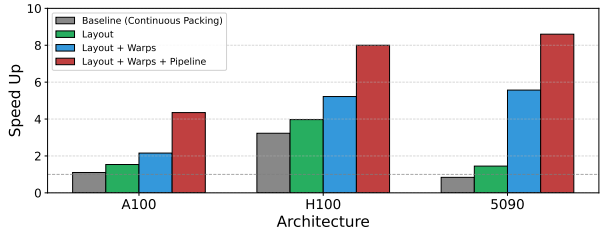


Fig. 16: Breakdown of BitDecoding optimizations across architectural generations.

while maintaining performance. Recent works explore 4-bit, 2-bit, and even 1-bit KV cache quantization, aiming to push the limits of compression. Methods like KIVI [18], Gear [13], and KVQuant [12] use per-channel quantization to handle key-value outliers, while RotateKV [27] applies rotation to smooth channel-wise distributions. Although effective at higher compression ratios, these methods lack efficient system implementations, leading to suboptimal performance.

b) Mixed-precision Matrix Multiplication: Low-bit weight and low-bit KV cache in LLMs create a unique requirement for mixed-precision matrix multiplication (mpGEMM), where one input matrix is in lower precision (e.g., INT4/2/1) while the other matrix remains in higher precision (e.g., FP16/8). Optimized kernels like Ladder [33] and Marlin [9] improve performance via layout transformations and efficient dequantization. However, these methods require pre-packing and pre-transforming weights, limiting applicability to low-bit KV cache in autoregressive decoding.

c) System Implementation for Low-bit KV Cache: KIVI [31] uses Triton with separate kernels for low-bit KV Cache implementation. Atom [37] integrates quantization within the preceding linear layer, while QServe [16] fuses quantization directly into FlashAttention kernels. However, they both rely on GEMV operations with fused multiply–add (FMA) instructions, missing Tensor Core acceleration.

VIII. Conclusion

In this paper, we introduce BitDecoding, a GPU-optimized computing framework supporting low-bit KV cache decoding with Tensor Cores. We effectively resolve the layout mismatches imposed by rigid hardware patterns and propose fine-grained optimizations to maximize computational utilization. Extensive evaluations demonstrate that BitDecoding achieves speedups of up to 8.6× on Blackwell, 8.9× on Hopper, 7.5× on Ada, and 4.8× on Ampere architectures compared to FP16 FlashDecoding-v2. Furthermore, on LLaMA-3.1-8B with a 128K sequence length, BitDecoding reduces single-batch decoding latency by 3× and improves serving throughput by 4× over state-of-the-art methods. By providing a high-performance system foundation, BitDecoding opens new avenues for algorithm-system co-design—paving the way for efficient, near-lossless test-time scaling in next-generation long-context models.

REFERENCES

[1] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.

[2] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, "Quarot: Outlier-free 4-bit inference in rotated llms," *Advances in Neural Information Processing Systems*, vol. 37, pp. 100 213–100 240, 2024.

[3] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3119–3137. [Online]. Available: https://aclanthology.org/2024.acl-long.172

[4] Y. Chang, K. Lo, T. Goyal, and M. Iyyer, "Booookscore: A systematic exploration of book-length summarization in the era of llms," *arXiv preprint arXiv:2310.00785*, 2023.

[5] N. Corporation, "Cutlass: Cuda templates for linear algebra subroutines and solvers," 2024, 3.6). [Online]. Available: https://github.com/NVIDIA/cutlass

[6] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] Y. Ding, L. L. Zhang, C. Zhang, Y. Xu, N. Shang, J. Xu, F. Yang, and M. Yang, "Longrope: Extending llm context window beyond 2 million tokens," *arXiv preprint arXiv:2402.13753*, 2024.

[8] G. Fan, M. Zhang, F. Zheng, S. Fan, T. Zhou, X. Deng, W. Tang, L. Kong, Y. Song, and S. Yan, "Warpdrive: Gpu-based fully homomorphic encryption acceleration leveraging tensor and cuda cores," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1187–1200.

[9] E. Frantar, R. L. Castro, J. Chen, T. Hoefler, and D. Alistarh, "Marlin: Mixed-precision auto-regressive parallel inference on large language models," *arXiv preprint arXiv:2408.11743*, 2024.

[10] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[11] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[12] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," *arXiv preprint arXiv:2401.18079*, 2024.

[13] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao, "Gear: An efficient kv cache compression recipefor near-lossless generative inference of llm," *arXiv preprint arXiv:2403.05527*, 2024.

[14] Y. J. Kim, R. Henry, R. Fahim, and H. H. Awadalla, "Who says elephants can't run: Bringing large scale moe models into cloud scale production," *arXiv preprint arXiv:2211.10017*, 2022.

[15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023. [Online]. Available: https://dl.acm.org/doi/10.1145/3600006.3613165

[16] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," *arXiv preprint arXiv:2405.04532*, 2024.

[17] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[18] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," *arXiv preprint arXiv:2402.02750*, 2024.

[19] W. Luo, R. Fan, Z. Li, D. Du, Q. Wang, and X. Chu, "Benchmarking and dissecting the nvidia hopper gpu architecture," *arXiv preprint arXiv:2402.13499*, 2024.

[20] NVIDIA and OpenAI, "OpenAI Triton on NVIDIA Blackwell Boosts AI Performance and Programmability," https://developer.nvidia.com/blog/openai-triton-on-nvidia-blackwell-boosts-ai-performance-and-programmability/, 2025, accessed: 2025-12-01.

[21] NVIDIA Corporation, "Nsight Compute - Get Started," 2025, accessed: 2025-03-11. [Online]. Available: https://developer.nvidia.com/tools-overview/nsight-compute/get-started

[22] OpenAI, "Openai o3-mini," 2025, accessed: 2025-02-14. [Online]. Available: https://openai.com/index/openai-o3-mini/

[23] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, "Yarn: Efficient context window extension of large language models," *arXiv preprint arXiv:2309.00071*, 2023.

[24] S. Sandokji, F. Essa, and M. Fadel, "A survey of techniques for warp scheduling in gpus," in *2015 IEEE Seventh International Conference on Intelligent Computing and Information Systems (ICICIS)*. IEEE, 2015, pp. 600–606.

[25] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," *Advances in Neural Information Processing Systems*, vol. 37, pp. 68 658–68 685, 2024.

[26] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.

[27] Z. Su, Z. Chen, W. Shen, H. Wei, L. Li, H. Yu, and K. Yuan, "Rotatekv: Accurate and robust 2-bit kv cache quantization for llms via outlier-aware adaptive rotations," *arXiv preprint arXiv:2501.16383*, 2025.

[28] L. Sun, J. Jiang, C. Deng, X. Wu, H. Zhang, L. Chen, L. Ni, and J. Wang, "Gta: Grouped-head latent attention," *arXiv preprint arXiv:2506.17286*, 2025.

[29] Q. Tao, W. Yu, and J. Zhou, "Asymkv: Enabling 1-bit quantization of kv cache with layer-wise asymmetric quantization configurations," *arXiv preprint arXiv:2410.13212*, 2024.

[30] G. Team, P. Georgiev, V. I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang *et al.*, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.

[31] P. Tillet, H.-T. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19.

[32] L. Wang, Y. Cheng, Y. Shi, Z. Tang, Z. Mo, W. Xie, L. Ma, Y. Xia, J. Xue, F. Yang *et al.*, "Tilelang: A composable tiled programming model for ai systems," *arXiv preprint arXiv:2504.17577*, 2025.

[33] L. Wang, L. Ma, S. Cao, Q. Zhang, J. Xue, Y. Shi, N. Zheng, Z. Miao, F. Yang, T. Cao *et al.*, "Ladder: Enabling efficient {Low-Precision} deep learning computing through hardware-aware tensor transformation," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 307–323.

[34] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[35] X. Yang, W. Wu, S. Feng, M. Wang, D. Wang, Y. Li, Q. Sun, Y. Zhang, X. Fu, and S. Poria, "Mm-bigbench: Evaluating multimodal models on multimodal content comprehension tasks," *arXiv preprint arXiv:2310.09036*, 2023.

[36] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, "Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 3304–3331, 2024.

[37] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate llm serving," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.